

Encryption at rest (9.5) — readiness audit

Date: 2026-08-09 **Question asked:** is the existing 9.5 code safe to run against the live database?

Answer: NO — not yet. Three things must be fixed first, and one product decision must be made.

Nothing was deployed, built, or run against the database to produce this. It is a code audit.

How much to trust this audit

Be aware of its limits before relying on it.

- Four independent audit passes **completed** (crypto core, backfill safety, the three known defects, rollout integrity). They produced **25 findings**.
- An adversarial verification pass was meant to try to *disprove* each finding. It got through **7 of 25** before the run died on a session limit. The final synthesis never ran.
- Of those 7: **5 were refuted or downgraded, 1 survived at High**, 1 was the re-audit itself.
- **18 findings were never verified at all.** They are leads, not confirmed defects. The refutation rate on the sample that was checked was high (5 of 6), so expect a meaningful fraction of the 18 to be noise — but that cuts both ways, and they are unexamined.

I separately verified the four highest-stakes structural items myself. Those are marked **[verified here]** and you can rely on them.

What is genuinely solid

Defect 1 — backfill infinite-looping against production: FIXED. `encrypt-backfill.mjs:165` uses keyset pagination with a cursor (`.gt(cursorCol, cursor)`) and `:173–176` throws if the cursor fails to advance. Progress no longer depends on the write clearing the `is null` filter. Composite-PK tables (`subscription_overrides`) take an offset path instead. The auditor ran it against a fake client with an all-NULL `monthly_limit` — the exact old hang — and it terminated cleanly.

Defect 3 — 4-byte auth tags accepted: FIXED. `crypto.js:85–88` rejects any tag that isn't exactly 16 bytes, belt-and-braces with `authTagLength` on `createDecipheriv`. A repo-wide grep confirms **one** decrypt call site, so there is no second unguarded path. `crypto.test.js:110–116` asserts the rejection. 19/19 pass.

Defect 2 — verification never read the database: PARTIALLY fixed. The per-row check is now real: it re-SELECTs by primary key and decrypts what Postgres actually returned. That part is sound. The *gate* built on top of it is not — see below.

The cryptographic core (AES-256-GCM, per-user HKDF keys, IV handling) drew no confirmed defect. Five findings against it were checked; all five were refuted or downgraded to low.

Must fix before the backfill runs

1. A row that fails verification is skipped forever [survived adversarial verification — High]

encrypt-backfill.mjs:108 commits the UPDATE **before** verifying at :114–140. If verification then fails — or merely hits a transient network blip on the read-back — the run aborts with the bad row already committed and non-NULL. The operator re-runs, as the script's own header invites. Line 162's idempotency filter (`.is(firstEnc, null)`) now **excludes that row permanently**. It prints “Backfill complete” and migration 013 drops the plaintext on the strength of that.

The verifier tried to refute this and couldn't. It downgraded Critical->High because the auditor's original mechanism (a truncating column) isn't reachable — the `_enc` columns are unbounded text. But it found a mundane path that is: a transient PostgREST timeout after the write lands.

Note both the spec (line 135) and the plan (line 508) **explicitly required this be handled**. Neither the script nor any migration implements it. There is no null-out-on-failure, no failed-row ledger, and no final re-verify pass.

Fix: null the `_enc` columns in a catch before rethrowing, *and* gate migration 013 on an independent full re-verify pass rather than on the backfill's own exit code.

2. Two tables added yesterday are outside the encryption scope entirely [verified here]

Migration 012 was written 2026-07-18. It covers 8 tables. `recurrences` and `special_groups` were created **yesterday** by Phase 10 — this gap is a direct consequence of my own work, and the encryption plan has no idea they exist.

<code>recurrences</code>	in migration 012: NO	in backfill JOBS: NO
<code>special_groups</code>	in migration 012: NO	in backfill JOBS: NO

`recurrences` holds amount and description — the same financial data as transactions. `special_groups` holds name — “September 2026 Paris holiday” tells you plenty on its own. Worse, `runRecurrences.js` inserts a transaction every night; post-013 that cron would write bare plaintext into a column the rest of the app treats as ciphertext.

Fix: extend migration 012 (as a new migration), the backfill JOBS list, and the route sweep to cover both tables and the cron insert path — before any of this runs.

3. There is no completeness gate on the irreversible drop

Several unverified findings converge on the same structural point, and it matches what I can see: nothing queries the database to prove every row is encrypted before 013 drops the plaintext. The plan's go/no-go is a manual UI click-through. Rows written by the live app *during* the rollout window would have NULL `_enc` and be dropped un-encrypted.

Fix: 013 must be gated on a query — `select count(*) ... where plaintext is not null and enc is null` returning 0 across every table — run immediately before the drop, in the same window.

The product decision you have to make

Encrypting transactions.description breaks two features, and one of them cannot be fixed by decrypting after fetch. [verified here]

- `server/routes/categories.js:89` runs `.ilike('description', '%term%')` — a **database-side** search. You cannot ILIKE a ciphertext. This is the merchant memory that rings the suggested category chip in Quick-Add (Task 6.9). It would silently return nothing, forever.
- `server/lib/subscriptions.js:20` derives `merchantKey` from `normaliseMerchant(description)`, and `subscription_overrides` uses that key as its primary key. Detection itself survives if the sweep decrypts before detecting — but it is coupled to plaintext in a way worth being deliberate about.

So you have a genuine choice:

(a) Encrypt descriptions and lose merchant memory — or rebuild it on a blind index (store an HMAC of the normalised merchant alongside the ciphertext, and search that). That’s real extra work.

(b) Leave description in plaintext, encrypt only the amounts. Weaker privacy — someone with dashboard access sees *what* you bought but not *how much*. Much simpler, and keeps every feature.

(c) Don’t do 9.5 at all right now. Defensible: see below.

The honest framing

The threat model in the spec is explicit — this stops **casual** viewing (you glancing at the Supabase dashboard, a backup file leaking). It is *not* protection from a determined attacker, because the server holds the key and must decrypt everything to function.

Against that: the work is a multi-session, irreversible, high-risk change to every route, on a live database with five other people’s data in it. Its plan has now produced **four** Critical-rated defects across two independent reviews (three found before, one confirmed here).

That is not an argument against doing it. It is an argument for doing it **deliberately**, with the fixes above landed first, and for being clear-eyed that “my friends’ data is encrypted” will be less true than it sounds while the server can read all of it on demand.

Recommended plan

Step 0 — decide the description question above. Everything downstream depends on it.

Step 1 — fix the backfill (no DB access needed). - Null out `_enc` on verification failure before rethrowing. - Reject unknown CLI flags so a typo’d `--dry-run` can’t perform a live run. - Add the first tests the backfill has ever had — termination, resumability, skip-protection. It currently has **zero**, despite being restructured specifically to be testable.

Step 2 — extend scope to recurrences and special_groups in a new migration, the JOBS list, and the sweep plan. Include the cron insert path.

Step 3 — build the completeness gate as a standalone script that queries the database, and make it the only thing that authorises 013.

Step 4 — verify the 18 unverified findings. Re-run the audit's verification pass. It died partway; the leads deserve a verdict before you bet on them being noise.

Step 5 — only then the runbook: key -> migration 012 -> dry run -> backfill -> sweep -> click-test -> gate -> 013.

Step 5 needs your key. Generate with `openssl rand -base64 32`, back it up in **two** places before it goes anywhere near `.env`. Lose it after 013 and every user's data is unreadable permanently.

Also outstanding, unrelated and much cheaper

Supabase's security advisor flags exactly one thing: ***leaked password protection is disabled*** (Authentication -> Passwords). RLS is on across all 10 tables with policies on each. That toggle is a minute's work and protects the front door your friends actually log in through — arguably better value per unit of risk than everything above.